

Siptic Manager

Manual completo de despliegue con Docker, Apache y Asterisk existente

Versión del documento: 1.0

Aplicación: Siptic Manager / Call Center Manager

Escenario: instalación limpia en un servidor que ya tiene Apache y Asterisk

Acceso propuesto: <https://empresa.com:4000>

Este procedimiento no instala, reemplaza ni reinicia Asterisk automáticamente. La integración telefónica utiliza archivos independientes, copias de seguridad y recarga opcional.

Siptic Manager	1
Manual completo de despliegue con Docker, Apache y Asterisk existente	1
1. Objetivo y resultado esperado	4
2. Principios de seguridad	4
3. Inventario previo obligatorio	4
4. Requisitos del servidor	5
4.1 Comprobar Apache y Asterisk	5
4.2 Comprobar puertos	5
4.3 Instalar Docker	5
4.4 Instalar ODBC PostgreSQL	6
5. Obtener y verificar el proyecto	6
6. Crear la configuración de producción	6
7. Levantar Docker y crear una base limpia	7
8. Configurar Apache en el puerto 4000	8
8.1 Validación sin cambios	8
8.2 Revisar VirtualHost generado	8
8.3 Aplicar	8
9. Configurar la integración Asterisk	9
9.1 Crear archivo privado	9
9.2 Revisar sin modificar	9
9.3 Escribir integración sin recargar Asterisk	9
9.4 Recarga controlada	9
10. Conectar el horario con una ruta de llamadas	10
11. Pruebas funcionales obligatorias	10
11.1 Aplicación	10
11.2 Llamada abierta	10
11.3 Cierre por falla técnica	11
11.4 Reentrenamiento	11
11.5 Estados inactivos	11
12. Respaldos	11
12.1 Manual	11
12.2 Automático	11
12.3 Restauración	11
13. Actualizaciones	12
14. Operación y diagnóstico	12
15. Rollback de Asterisk	12
16. Lista de aceptación para producción	13

17. Preguntas frecuentes y solución de problemas	13
¿Docker debe incluir Asterisk?	13
¿Se modifica la base actual de Asterisk?	13
¿Asterisk puede consultar una base dentro de Docker?.....	13
¿PostgreSQL queda expuesto a Internet?.....	14
docker: command not found	14
permission denied al usar Docker	14
El puerto 4000 está ocupado	14
Apache no inicia después de configurar.....	14
Error de certificado al entrar por :4000.....	14
La aplicación redirige repetidamente a HTTPS.....	14
Error CSRF al iniciar sesión	15
Django aparece unhealthy	15
PostgreSQL aparece unhealthy	15
role callcenter_app already exists.....	15
isql: Data source name not found	15
ODBC conecta pero Asterisk no muestra la conexión	16
ODBC_SIPTIC_CALLCENTER_STATUS no existe	16
El dialplan no encuentra siptic-validar-horario	16
El CallCenter siempre aparece cerrado	16
El horario está desplazado una hora	16
Se escucha vm-goodbye en vez del audio elegido.....	16
El usuario ve CallCenters de otro cliente	17
Dos usuarios editaron el mismo horario.....	17
¿Cómo cambiar el dominio o puerto?.....	17
¿Cómo cambiar una contraseña PostgreSQL?	17
¿Cómo retirar la aplicación sin borrar datos?	17
¿Cómo borrar definitivamente una instalación de prueba?.....	17
18. Información para soporte	17
19. Regenerar este manual en PDF.....	18

1. Objetivo y resultado esperado

Al terminar este manual estarán funcionando:

- Django y Gunicorn dentro de Docker.
- PostgreSQL 17 dentro de Docker con una base nueva.
- Apache del servidor publicando la aplicación mediante HTTPS en el puerto 4000.
- Asterisk consultando PostgreSQL por ODBC a través de `127.0.0.1:55432`.
- Audios de cierre dentro de una carpeta exclusiva de Asterisk.
- Respaldos manuales y automáticos.
- Comandos de diagnóstico, actualización y restauración.

La arquitectura final será:

```
Usuario
| https://empresa.com:4000
▼
Apache existente
| http://127.0.0.1:14000
▼
Django + Gunicorn (Docker)
| red privada Docker, db:5432
▼
PostgreSQL 17 (Docker)
▲
| 127.0.0.1:55432, solo lectura
|
Asterisk existente en el host
```

2. Principios de seguridad

- Gunicorn no se publica directamente en Internet.
- PostgreSQL solamente escucha en `127.0.0.1` del servidor.
- Asterisk usa un usuario PostgreSQL exclusivo de solo lectura.
- Django usa otro usuario sin privilegios de superusuario.
- El administrador PostgreSQL tiene una contraseña diferente.
- Los secretos permanecen en archivos con permisos `600` y no entran en Git.
- El instalador no modifica endpoints, troncales, colas, DIDs ni rutas existentes.
- Ningún script elimina volúmenes Docker automáticamente.
- Toda restauración exige confirmación y crea un respaldo previo.

3. Inventario previo obligatorio

Registrar antes de comenzar:

Dato	Ejemplo	Valor real
Dominio	<code>empresa.com</code>	

Dato	Ejemplo	Valor real
Puerto web	4000	
IP del servidor	192.0.2.20	
Ruta del certificado	/etc/letsencrypt/live/empresa.com/fullchain.pem	
Ruta de llave	/etc/letsencrypt/live/empresa.com/privkey.pem	
Distribución	Debian 13	
Usuario de despliegue	siptic	
Ruta del proyecto	/opt/callcenter_manager	
Ruta configuración Asterisk	/etc/asterisk	
Ruta audios Asterisk	/var/lib/asterisk/sounds	

Guardar además un respaldo externo de:

```
sudo tar -czf /root/asterisk_antes_siptic.tar.gz \
/etc/asterisk /etc/odbc.ini /var/lib/asterisk/sounds
```

4. Requisitos del servidor

4.1 Comprobar Apache y Asterisk

```
apache2ctl -S
apache2ctl -M
asterisk -rx "core show version"
asterisk -rx "module show like func_odbc"
```

No continuar si Asterisk no responde correctamente antes del despliegue.

4.2 Comprobar puertos

```
sudo ss -ltnp | grep -E ':(4000|14000|55432)\b'
```

Antes de instalar, los tres puertos deberían estar libres. Después:

- 4000 pertenecerá a Apache.
- 14000 pertenecerá a Docker y escuchará solo en loopback.
- 55432 pertenecerá a Docker y escuchará solo en loopback.

4.3 Instalar Docker

Instalar Docker Engine y el complemento Compose desde la documentación oficial:

```
https://docs.docker.com/engine/install/
```

Verificar:

```
docker --version
docker compose version
sudo docker run --rm hello-world
```

4.4 Instalar ODBC PostgreSQL

En Debian/Ubuntu:

```
sudo apt update
sudo apt install unixodbc odbc-postgresql
odbcinst -q -d
```

Debe aparecer un driver como `PostgreSQL Unicode`. Si el nombre es distinto se configurará posteriormente en `asterisk/.env`.

5. Obtener y verificar el proyecto

```
sudo mkdir -p /opt/callcenter_manager
sudo chown "$USER":"$USER" /opt/callcenter_manager
git clone URL_DEL_REPOSITORIO /opt/callcenter_manager
cd /opt/callcenter_manager
git checkout develop-m
git log -2 --oneline
git status --short
```

Se esperan al menos estos commits:

```
1f11f7a build: prepara despliegue Docker con Apache y Asterisk
8824051 feat: consolida gestion y estado horario para Asterisk
```

`git status --short` no debería mostrar modificaciones.

6. Crear la configuración de producción

```
cd /opt/callcenter_manager
./deploy/scripts/init-env.sh empresa.com 4000
chmod 600 deploy/.env.production
nano deploy/.env.production
```

El comando genera automáticamente cuatro secretos diferentes. Revisar:

```
DJANGO_ALLOWED_HOSTS=empresa.com,127.0.0.1
DJANGO_CSRF_TRUSTED_ORIGINS=https://empresa.com:4000
DB_NAME=callcenter_manager
DB_USER=callcenter_app
ASTERISK_DB_USER=asterisk_schedule_reader
ASTERISK_DB_PORT=55432
APP_INTERNAL_PORT=14000
APP_EXTERNAL_PORT=4000
APACHE_CERTIFICATE_FILE=/etc/letsencrypt/live/empresa.com/fullchain.pem
APACHE_CERTIFICATE_KEY_FILE=/etc/letsencrypt/live/empresa.com/privkey.pem
```

No copiar la contraseña administrativa como contraseña de Django o Asterisk.

Validar:

```
./deploy/scripts/check.sh
```

7. Levantar Docker y crear una base limpia

```
./deploy/scripts/install.sh
```

Este comando:

1. Valida Docker, variables y puertos.
2. Construye la imagen Django.
3. Crea PostgreSQL 17.
4. Crea `callcenter_app` y `asterisk_schedule_reader`.
5. Aplica migraciones.
6. Crea las vistas para Asterisk.
7. Concede acceso de solo lectura.
8. Ejecuta `collectstatic`.
9. Espera los health checks.
10. Solicita el primer superusuario si no existe.

Comprobar:

```
./deploy/scripts/status.sh
docker compose \
  --env-file deploy/.env.production \
  -f deploy/compose.yaml ps
```

No ejecutar:

```
docker compose down --volumes
```

Ese comando eliminaría la base persistente.

8. Configurar Apache en el puerto 4000

8.1 Validación sin cambios

```
sudo -E SIPTIC_ENV_FILE="$PWD/deploy/.env.production" \  
./deploy/apache/configure.sh --check
```

8.2 Revisar VirtualHost generado

```
sudo -E SIPTIC_ENV_FILE="$PWD/deploy/.env.production" \  
./deploy/apache/configure.sh --dry-run
```

Debe contener:

- `VirtualHost *:4000`.
- `ServerName empresa.com`.
- Certificado correcto.
- Proxy hacia `127.0.0.1:14000`.
- `X-Forwarded-Proto https`.

8.3 Aplicar

```
sudo -E SIPTIC_ENV_FILE="$PWD/deploy/.env.production" \  
./deploy/apache/configure.sh --apply
```

El script habilita únicamente `ssl`, `proxy`, `proxy_http` y `headers`, valida con `apache2ctl configtest` y recarga Apache solo si la configuración es válida.

Abrir firewall, por ejemplo con UFW:

```
sudo ufw allow 4000/tcp  
sudo ufw status
```

Pruebas:

```
curl -I https://empresa.com:4000/health/  
curl -I https://empresa.com:4000/login/
```

La web existente puede enlazar a:

```
<a href="https://empresa.com:4000/">Siptic Manager</a>
```


9. Configurar la integración Asterisk

9.1 Crear archivo privado

```
cp asterisk/.env.example asterisk/.env
chmod 600 asterisk/.env
nano asterisk/.env
```

Copiar `ASTERISK_DB_PASSWORD` desde `deploy/.env.production`. Configuración habitual:

```
ASTERISK_DB_HOST=127.0.0.1
ASTERISK_DB_PORT=55432
ASTERISK_DB_NAME=callcenter_manager
ASTERISK_DB_USER=asterisk_schedule_reader
ASTERISK_ODBC_DSN=callcenter_pg
ASTERISK_ODBC_DRIVER=PostgreSQL Unicode
ASTERISK_CONFIG_DIR=/etc/asterisk
ASTERISK_SOUND_DIR=/var/lib/asterisk/sounds/custom/callcenter_manager
```

9.2 Revisar sin modificar

```
sudo -E SIPTIC_ASTERISK_ENV="$PWD/asterisk/.env" \
./asterisk/integrate.sh --check

sudo -E SIPTIC_ASTERISK_ENV="$PWD/asterisk/.env" \
./asterisk/integrate.sh --dry-run
```

9.3 Escribir integración sin recargar Asterisk

```
sudo -E SIPTIC_ASTERISK_ENV="$PWD/asterisk/.env" \
./asterisk/integrate.sh --apply
```

Se crean únicamente:

```
/etc/asterisk/siptic_manager_res_odbc.conf
/etc/asterisk/siptic_manager_func_odbc.conf
/etc/asterisk/siptic_manager_dialplan.conf
/var/lib/asterisk/sounds/custom/callcenter_manager/*.wav
```

Además se añade un bloque identificado en `/etc/odbc.ini` y una línea `#include` en cada archivo principal. Los respaldos quedan en:

```
/var/backups/siptic-manager/asterisk/
```

9.4 Recarga controlada

Después de revisar archivos y confirmar la conexión:

```
sudo -E SIPTIC_ASTERISK_ENV="$PWD/asterisk/.env" \  
./asterisk/integrate.sh --apply --reload
```

No reinicia Asterisk. Recarga `res_odbc`, `func_odbc` y el dialplan.

Comprobar:

```
asterisk -rx "odbc show"  
asterisk -rx "dialplan show siptic-validar-horario"  
asterisk -rx "module show like func_odbc"
```

10. Conectar el horario con una ruta de llamadas

Este paso es manual porque cada central tiene contextos, DIDs y rutas diferentes. En el punto correcto del dialplan se llama:

```
same => n,Gosub(siptic-validar-horario,s,1(pas_aba_cla))
```

Reemplazar `pas_aba_cla` por el código creado en la interfaz.

Antes de modificar una ruta:

```
asterisk -rx "dialplan show CONTEXT0"  
sudo cp -a /etc/asterisk/extensions.conf \  
/root/extensions.conf.antes_siptic
```

No insertar el `Gosub` en todas las llamadas globalmente. Debe colocarse solo en la ruta del CallCenter correspondiente.

11. Pruebas funcionales obligatorias

11.1 Aplicación

- Iniciar sesión como superusuario.
- Crear un cliente.
- Crear un CallCenter con código técnico estable.
- Crear un usuario normal y asignarlo al cliente.
- Verificar que solo vea sus CallCenters.
- Crear horario y revisar auditoría.

11.2 Llamada abierta

1. Crear un intervalo que incluya la hora actual.
2. Guardar como apertura.
3. Llamar al DID.

4. Confirmar que la llamada continúe hacia la cola o destino existente.

11.3 Cierre por falla técnica

1. Retirar el intervalo actual o cerrar el día.
2. Elegir “Falla técnica”.
3. Llamar.
4. Confirmar audio y finalización.

11.4 Reentrenamiento

Repetir usando “Reentrenamiento de personal”.

11.5 Estados inactivos

- Desactivar CallCenter y probar llamada.
- Reactivarlo.
- Desactivar cliente y probar llamada.
- Confirmar que PostgreSQL responda `is_enabled=false`.

12. Respaldos

12.1 Manual

```
./deploy/scripts/backup.sh  
ls -lh deploy/backups/
```

12.2 Automático

```
sudo ./deploy/systemd/install.sh  
systemctl status siptic-manager-backup.timer  
systemctl list-timers siptic-manager-backup.timer
```

El temporizador ejecuta un respaldo diario. Debe copiarse periódicamente a otro servidor o almacenamiento; un respaldo en el mismo disco no protege frente a pérdida del servidor.

12.3 Restauración

```
./deploy/scripts/restore.sh \  
deploy/backups/callcenter_manager_20260722T120000Z.dump
```

El comando exige escribir `RESTAURAR`, genera un respaldo previo, detiene temporalmente Django, restaura y vuelve a levantarlo.

13. Actualizaciones

```
cd /opt/callcenter_manager
git status --short
git pull --ff-only
./deploy/scripts/update.sh
```

El script realiza respaldo, reconstruye la imagen, aplica migraciones y comprueba el estado.

Nunca actualizar con archivos locales sin commit o sin respaldo.

14. Operación y diagnóstico

```
./deploy/scripts/status.sh
docker compose --env-file deploy/.env.production \
-f deploy/compose.yaml logs -f web
docker compose --env-file deploy/.env.production \
-f deploy/compose.yaml logs -f db
```

Estado Apache:

```
apache2ctl configtest
systemctl status apache2
journalctl -u apache2 --since "30 minutes ago"
```

Estado Asterisk:

```
asterisk -rx "odbc show"
asterisk -rx "dialplan show siptic-validar-horario"
asterisk -rx "core show channels"
```

Vista PostgreSQL desde Django:

```
docker compose --env-file deploy/.env.production \
-f deploy/compose.yaml exec web \
python manage.py dbshell
```

```
SELECT * FROM schedules_asterisk_callcenter_status;
```

15. Rollback de Asterisk

Para retirar únicamente la integración administrada por Siptic Manager:

```
sudo -E SIPTIC_ASTERISK_ENV="$PWD/asterisk/.env" \
./asterisk/integrate.sh --rollback
```

Esto elimina archivos dedicados, audios dedicados, bloque ODBC y líneas `#include`. No toca rutas, endpoints, colas ni troncales.

Después validar antes de recargar:

```
asterisk -rx "dialplan reload"
```

16. Lista de aceptación para producción

- [] Docker y Compose instalados.
- [] Dominio resuelve hacia el servidor.
- [] Certificado válido.
- [] Puerto 4000 permitido en firewall.
- [] Puertos 14000 y 55432 solo en loopback.
- [] `.env.production` y `asterisk/.env` con permisos 600.
- [] Health checks saludables.
- [] Superusuario creado.
- [] Apache responde por HTTPS.
- [] ODBC conectado.
- [] Ambos audios probados.
- [] Llamada abierta probada.
- [] Cierre probado.
- [] Cliente y CallCenter inactivos probados.
- [] Auditoría verificada.
- [] Respaldo creado.
- [] Restauración probada en ambiente controlado.
- [] Copia externa del respaldo configurada.

17. Preguntas frecuentes y solución de problemas

¿Docker debe incluir Asterisk?

No. Asterisk continúa instalado en el host. Docker contiene únicamente Django y PostgreSQL.

¿Se modifica la base actual de Asterisk?

No. Se crea una base PostgreSQL independiente llamada `callcenter_manager`.

¿Asterisk puede consultar una base dentro de Docker?

Sí. Docker publica PostgreSQL en `127.0.0.1:55432` ; para Asterisk se comporta como un servicio local.

¿PostgreSQL queda expuesto a Internet?

No. Verificar:

```
sudo ss -ltnp | grep 55432
```

Debe mostrar `127.0.0.1:55432`, nunca `0.0.0.0:55432`.

`docker: command not found`

Docker no está instalado o no está en `PATH`. Instalar Docker Engine y Compose, cerrar sesión y volver a entrar si se añadió el usuario al grupo `docker`.

`permission denied` al usar Docker

Ejecutar temporalmente con `sudo` o añadir el usuario autorizado al grupo Docker. Pertenecer al grupo Docker equivale prácticamente a acceso root y debe limitarse.

El puerto 4000 está ocupado

Identificar el proceso:

```
sudo ss -ltnp | grep ':4000'
```

Elegir otro puerto, por ejemplo 4443, y volver a crear o ajustar `.env.production`. No detener un servicio desconocido.

Apache no inicia después de configurar

```
apache2ctl configtest
journalctl -u apache2 -n 100 --no-pager
```

Revisar certificados, duplicación de `Listen` y módulos. El script no recarga Apache si `configtest` falla.

Error de certificado al entrar por `:4000`

El certificado valida el dominio, no el puerto. Comprobar que se usa el mismo dominio incluido en el certificado y que las rutas del certificado sean correctas.

La aplicación redirige repetidamente a HTTPS

Apache debe enviar:

```
RequestHeader set X-Forwarded-Proto "https"
```

Y Django debe tener:

```
DJANGO_USE_X_FORWARDED_PROTO=True
```

Error CSRF al iniciar sesión

Comprobar que el origen incluya protocolo y puerto exactos:

```
DJANGO_CSRF_TRUSTED_ORIGINS=https://empresa.com:4000
```

Después recrear el contenedor web:

```
docker compose --env-file deploy/.env.production \
-f deploy/compose.yaml up -d --force-recreate web
```

Django aparece unhealthy

```
docker compose --env-file deploy/.env.production \
-f deploy/compose.yaml logs --tail 150 web db
```

Revisar conexión PostgreSQL, migraciones, `ALLOWED_HOSTS` y secretos.

PostgreSQL aparece unhealthy

```
docker compose --env-file deploy/.env.production \
-f deploy/compose.yaml logs --tail 150 db
```

En la primera ejecución puede tardar algunos segundos. Si el volumen quedó parcialmente inicializado por una prueba fallida, no eliminarlo sin revisar si contiene datos.

role callcenter_app already exists

Normalmente indica que se reutilizó un volumen parcialmente inicializado. Revisar los logs. En una instalación realmente nueva se puede retirar el volumen únicamente después de confirmar que no contiene información necesaria.

isql: Data source name not found

Revisar:

```
odbcinst -q -s
odbcinst -q -d
grep -A8 '\[callcenter_pg\]' /etc/odbc.ini
```

El valor `ASTERISK_ODBC_DRIVER` debe coincidir exactamente con el listado de `odbcinst -q -d`.

ODBC conecta pero Asterisk no muestra la conexión

```
asterisk -rx "module reload res_odbc.so"
asterisk -rx "odbc show"
```

Revisar el `#include siptic_manager_res_odbc.conf` y los permisos de lectura del usuario Asterisk.

ODBC_SIPTIC_CALLCENTER_STATUS no existe

```
asterisk -rx "module reload func_odbc.so"
asterisk -rx "module show like func_odbc"
```

Revisar `siptic_manager_func_odbc.conf` y su `#include`.

El dialplan no encuentra siptic-validar-horario

```
asterisk -rx "dialplan reload"
asterisk -rx "dialplan show siptic-validar-horario"
```

Revisar `#include siptic_manager_dialplan.conf` en `extensions.conf`.

El CallCenter siempre aparece cerrado

Comprobar:

- Cliente activo.
- CallCenter activo.
- Zona horaria correcta.
- Día de la semana correcto.
- Intervalo que incluya la hora actual.

Consultar directamente:

```
SELECT codename, timezone, is_enabled, is_open, playback_file
FROM schedules_asterisk_callcenter_status;
```

El horario está desplazado una hora

Verificar la zona IANA del CallCenter, por ejemplo `America/Bogota`. No usar abreviaciones como `COT`, `EST` o `GMT-5`.

Se escucha vm-goodbye en vez del audio elegido

El archivo configurado no fue encontrado. Revisar:

```
ls -l /var/lib/asterisk/sounds/custom/callcenter_manager/
file /var/lib/asterisk/sounds/custom/callcenter_manager/*.wav
```


Los archivos deben ser mono, PCM, 8 kHz.

El usuario ve CallCenters de otro cliente

No debería ocurrir. Desactivar inmediatamente el acceso, guardar evidencia y revisar la asignación del usuario. La aplicación filtra por `client_id`; no corregirlo solo ocultando elementos con CSS.

Dos usuarios editaron el mismo horario

La segunda edición recibe HTTP 409 y debe recargar. Es una protección contra sobrescrituras silenciosas.

¿Cómo cambiar el dominio o puerto?

Actualizar `DJANGO_ALLOWED_HOSTS`, `DJANGO_CSRF_TRUSTED_ORIGINS`, `APP_EXTERNAL_PORT` y las rutas del certificado. Volver a aplicar Apache y recrear `web`.

¿Cómo cambiar una contraseña PostgreSQL?

Cambiarla requiere actualizar PostgreSQL y el `.env` correspondiente de forma coordinada. Cambiar solamente el archivo no modifica una base ya inicializada.

¿Cómo retirar la aplicación sin borrar datos?

```
docker compose --env-file deploy/.env.production \
-f deploy/compose.yaml down
```

No añadir `--volumes`.

¿Cómo borrar definitivamente una instalación de prueba?

Solo después de respaldar y confirmar el nombre del proyecto y volumen. La eliminación de volúmenes es irreversible y no está automatizada deliberadamente.

18. Información para soporte

Al reportar un problema adjuntar, ocultando secretos:

```
git log -2 --oneline
./deploy/scripts/status.sh
docker compose --env-file deploy/.env.production \
-f deploy/compose.yaml logs --tail 200 web db
apache2ctl -S
asterisk -rx "odbc show"
asterisk -rx "dialplan show siptic-validar-horario"
```

Nunca adjuntar `.env.production`, `asterisk/.env`, contraseñas, llaves privadas o dumps sin cifrar.

19. Regenerar este manual en PDF

El PDF está versionado junto con su fuente editable. Si cambia el proceso de despliegue, se puede regenerar así desde la raíz del repositorio:

```
python3 -m venv .venv-docs
.venv-docs/bin/pip install -r docs/requirements.txt
.venv-docs/bin/python docs/generate_deployment_pdf.py
```

El resultado será `docs/Manual_Despliegue_Siptic_Manager.pdf`. El entorno `.venv-docs` sirve únicamente para generar documentación y no debe copiarse al servidor de producción.